K8S-99: Best Practice Summary

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 07/15/2026

Overview

This notebook consolidates every actionable best practice for Dynatrace Kubernetes monitoring and DynaKube configuration extracted from the K8S series (notebooks 01-13). Each practice specifies the exact setting, value, priority, and category. Use this as a definitive checklist for new deployments and audits of existing environments.

Operator Version: 1.9.0 | **DynaKube API:** `dynatrace.com/v1beta5` (baseline) or `v1beta6` (newer) | **Helm:** `oci://public.ecr.aws/dynatrace/dynatrace-operator --version 1.10.0` (pin the version you have validated in your estate)

> **Token currency note:** Platform Tokens (`dt0s16`) are the recommended choice for new tenants per Dynatrace SaaS sprint-1.337+; the Operator itself accepts platform tokens from **Operator 1.10.0** (July 15, 2026) — Classic API Tokens (`dt0c01`) continue to be accepted and remain the working path on earlier Operator versions. See K8S-02 §Prerequisites.
>
> **Operator 1.10.0 upgrade notes (July 15, 2026):** the ActiveGate `TopologySpreadConstraint` default changed from `DoNotSchedule` to `ScheduleAnyway` — expect an **ActiveGate restart on upgrade**. DynaKube `v1beta3` was removed in Operator 1.9.0 and `v1beta4` is deprecated as of 1.10.0 — target `v1beta6` for new DynaKubes (`v1beta5` remains accepted). The `gke-autopilot.yaml` release artifact is removed — GKE Autopilot deploys via Helm.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail and Kubernetes monitoring enabled
Kubernetes Cluster	v1.24+
Helm	v3.x
Dynatrace Operator	v1.9.0 (April 2026) via `oci://public.ecr.aws/dynatrace/dynatrace-operator`
Knowledge	K8S-01 through K8S-13

1. Deployment Mode Selection

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Use `cloudNativeFullStack` for new deployments	`spec.oneAgent.cloudNativeFullStack: {}`	**Critical**	Deployment
2	Do NOT use `classicFullStack` for new deployments	Remove `spec.oneAgent.classicFullStack`	**Critical**	Deployment
3	Use `applicationMonitoring` when infra visibility is not needed	`spec.oneAgent.applicationMonitoring: { useCSIDriver: true }`	Recommended	Deployment
4	Use `hostMonitoring` for infra-only clusters	`spec.oneAgent.hostMonitoring: {}`	Optional	Deployment
5	Omit `oneAgent` entirely for infrastructure-only monitoring alongside other APM tools	Only configure `spec.activeGate`	Recommended	Deployment

> **classicFullStack** is legacy. It uses host-path mounts and shares a single OneAgent across all pods. `cloudNativeFullStack` injects code modules via webhook and CSI driver, enabling independent app/infra monitoring with no privileged containers for applications.

2. Operator Installation

#	Best Practice	Recommended Setting/Value	Priority	Category
6	Install operator via Helm OCI	`helm install dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-operator --version 1.10.0 --namespace dynatrace --create-namespace --wait`	**Critical**	Installation
7	Enable CSI driver	`csidriver.enabled: true` in Helm values	**Critical**	Installation
8	Set platform explicitly	`platform: "kubernetes"` or `platform: "openshift"`	Recommended	Installation
9	Create dedicated namespace	`kubectl create namespace dynatrace`	**Critical**	Installation
10	Create API token secret before applying DynaKube	`kubectl create secret generic dynakube --namespace dynatrace --from-literal=apiToken=<TOKEN> --from-literal=dataIngestToken=<TOKEN>`	**Critical**	Installation

Required Token Scopes

Operator Token: `activeGateTokenManagement.create`, `entities.read`, `settings.read`, `settings.write`, `InstallerDownload`

Data Ingest Token: `metrics.ingest`, `logs.ingest`

3. DynaKube Core Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
11	Use API version v1beta5 or v1beta6	`apiVersion: dynatrace.com/v1beta5`	**Critical**	Configuration
12	Set `apiUrl` to your SaaS tenant	`spec.apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api`	**Critical**	Configuration
13	Add control-plane tolerations to OneAgent	`tolerations: [{effect: NoSchedule, key: node-role.kubernetes.io/master, operator: Exists}, {effect: NoSchedule, key: node-role.kubernetes.io/control-plane, operator: Exists}]`	Recommended	Configuration
14	Set `nodeSelector` for linux-only	`nodeSelector: {kubernetes.io/os: linux}`	Recommended	Configuration
15	Set OneAgent resource requests and limits	`requests: {cpu: 100m, memory: 256Mi}`, `limits: {cpu: 300m, memory: 512Mi}`	Recommended	Configuration
16	Set `networkZone` for routing isolation	`spec.networkZone: production`	Optional	Configuration
17	Set `hostGroup` for logical grouping	`spec.oneAgent.hostGroup: production`	Optional	Configuration

Minimal Production DynaKube

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api
  metadataEnrichment:
    enabled: true
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api
```

4. ActiveGate Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
18	Enable `kubernetes-monitoring` capability	`spec.activeGate.capabilities: [kubernetes-monitoring, routing]`	**Critical**	ActiveGate
19	Add `dynatrace-api` capability	Add `dynatrace-api` to capabilities list	Recommended	ActiveGate

#	Best Practice	Recommended Setting/Value	Priority	Category
20	Set ActiveGate replicas >= 2 for production	`spec.activeGate.replicas: 2`	**Critical**	ActiveGate
21	Set ActiveGate resource limits	`limits: {cpu: 1000m, memory: 2Gi}` for medium clusters	**Critical**	ActiveGate
22	Add zone-aware topology spread	`topologySpreadConstraints: [{maxSkew: 1, topologyKey: topology.kubernetes.io/zone, whenUnsatisfiable: ScheduleAnyway}]`	Recommended	ActiveGate

ActiveGate Sizing Reference

Cluster Size	Nodes	CPU Limit	Memory Limit	Replicas
Small	1-10	500m	1Gi	1
Medium	10-50	1000m	2Gi	2
Large	50-100	2000m	4Gi	2
Enterprise	100+	4000m	8Gi	3+

5. Namespace and Injection Control

#	Best Practice	Recommended Setting/Value	Priority	Category
23	Use `namespaceSelector` to control injection scope	`spec.oneAgent.cloudNativeFullStack.namespaceSelector.matchLabels: {dynatrace-injection: enabled}`	Recommended	Injection
24	Do not use the `default` namespace for workloads	Enforce via policy	Recommended	Namespace
25	Apply consistent labels to all namespaces	`team`, `env`, `cost-center` labels	Recommended	Namespace
26	Use `matchExpressions` to exclude system namespaces	`operator: NotIn, values: [kube-system, newrelic, datadog]`	Recommended	Injection
27	Use pod annotation to disable injection for specific pods	`oneagent.dynatrace.com/inject: "false"`	Optional	Injection
28	Limit injected technologies when needed	`oneagent.dynatrace.com/technologies: "java,nodejs"`	Optional	Injection
29	Set ResourceQuotas on every namespace	`requests.cpu`, `requests.memory`, `limits.cpu`, `limits.memory`, `pods`	Recommended	Namespace
30	Set LimitRanges for default container limits	`default: {cpu: 500m, memory: 512Mi}`, `defaultRequest: {cpu: 100m, memory: 128Mi}`	Recommended	Namespace

6. Feature Flags and Annotations

#	Best Practice	Recommended Setting/Value	Priority	Category
31	Enable Kubernetes app detection (unofficial — not in Dynatrace docs; verify before using)	`feature.dynatrace.com/k8s-app-enabled: "true"` on DynaKube metadata	Recommended	Feature Flag
32	Enable version detection from K8s labels	`feature.dynatrace.com/label-version-detection: "true"`	Recommended	Feature Flag
33	Set injection failure policy to `fail` in non-prod	`feature.dynatrace.com/injection-failure-policy: "fail"`	Recommended	Feature Flag
34	Set injection failure policy to `silent` in prod	`feature.dynatrace.com/injection-failure-policy: "silent"`	**Critical**	Feature Flag
35	Use opt-in mode for multi-tool coexistence	`feature.dynatrace.com/automatic-injection: "false"` + `namespaceSelector`	Recommended	Feature Flag
36	Apply `app.kubernetes.io/version` label to all deployments	`app.kubernetes.io/version: "1.2.3"` in pod template labels	Recommended	Labels
37	Apply all Kubernetes recommended labels	`app.kubernetes.io/name`, `app.kubernetes.io/component`, `app.kubernetes.io/part-of`	Recommended	Labels

7. Metadata Enrichment

#	Best Practice	Recommended Setting/Value	Priority	Category
38	Enable metadata enrichment	`spec.metadataEnrichment.enabled: true`	**Critical**	Enrichment
39	Use settings-based enrichment (not pod annotations)	Configure rules in Settings > Cloud and virtualization > Kubernetes telemetry enrichment	**Critical**	Enrichment
40	Do NOT mix settings-based and pod-annotation enrichment	Use one method only	**Critical**	Enrichment
41	Create enrichment rules for `team`, `env`, `cost-center` labels	Metadata type: Label, Key: `team`, Prefix: `k8s.`	Recommended	Enrichment
42	Configure enrichment rules before deploying DynaKube	Rules propagate immediately when DynaKube is first applied	Recommended	Enrichment
43	Allow 45 minutes for enrichment propagation after rule changes	New/modified rules take up to 45 min on existing deployments	Recommended	Enrichment
44	Label namespaces with cost allocation metadata	`cost-center`, `budget-owner` labels on Namespace objects	Recommended	Enrichment

8. Log Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
45	Enable log monitoring with empty object	`spec.logMonitoring: {}`	Recommended	Logs
46	Configure log monitoring resources under `templates`	`spec.templates.logMonitoring.resources: {limits: {cpu: 500m, memory: 512Mi}}`	**Critical**	Logs
47	Never nest properties inside `spec.logMonitoring`	`spec.logMonitoring: {}` only; resources/tolerations go under `spec.templates.logMonitoring`	**Critical**	Logs

#	Best Practice	Recommended Setting/Value	Priority	Category
48	Add control-plane tolerations to log monitoring	`spec.templates.logMonitoring.tolerations: [{effect: NoSchedule, key: node-role.kubernetes.io/control-plane, operator: Exists}]`	Recommended	Logs
49	Use OpenPipeline to filter debug logs before storage	Drop `loglevel == "DEBUG"` in processing pipeline	Recommended	Logs
50	Route logs to separate Grail buckets by environment	OpenPipeline route rules based on `k8s.env` enriched label	Optional	Logs

9. OTel Collector and Telemetry Ingest

#	Best Practice	Recommended Setting/Value	Priority	Category
51	Pin OTel Collector image to a specific version (current: `0.48.0`, May 2026)	`spec.templates.otelCollector.imageRef.tag: "0.48.0"`	**Critical**	OTel
52	Never use `latest` tag for OTel Collector	Always specify explicit version tag	**Critical**	OTel
53	Set OTel Collector resource limits	`limits: {cpu: 500m, memory: 512Mi}` for staging; `{cpu: 1000m, memory: 1Gi}` for production	Recommended	OTel
54	Configure `telemetryIngest` with required protocols	`spec.telemetryIngest.protocols: [otlp]` (add `statsd`, `jaeger`, `zipkin` as needed)	Recommended	OTel
55	Use StatsD via OTel Collector on K8s (not OneAgent StatsD daemon)	Deploy OTel Collector with StatsD receiver; OneAgent StatsD is not available on K8s	**Critical**	OTel
56	Deploy StatsD OTel Collector in a dedicated `monitoring` namespace	Cluster-wide Deployment; apps reference via FQDN `otel-collector-statsd.monitoring.svc.cluster.local:8125`	Recommended	OTel

10. CSI Driver Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
57	Enable CSI driver in Helm values	`csidriver.enabled: true`	**Critical**	CSI
58	Set resource limits on provisioner container	`provisioner.resources.limits: {cpu: 500m, memory: 256Mi}`	**Critical**	CSI
59	Set resource limits on all 5 CSI containers	`csiInit`, `server`, `provisioner`, `registrar`, `livenessprobe`	Recommended	CSI

CSI Driver Container Resource Reference

Container	CPU Request	CPU Limit	Memory Request	Memory Limit
`csiInit`	50m	100m	100Mi	128Mi
`server`	50m	100m	100Mi	128Mi
`provisioner`	300m	500m	100Mi	256Mi
`registrar`	20m	50m	30Mi	64Mi
`livenessprobe`	20m	50m	30Mi	64Mi

> **Warning:** Default Helm `values.yaml` may be missing limits for the `provisioner` container. Always add them explicitly.

11. Resource Sizing

#	Best Practice	Recommended Setting/Value	Priority	Category
60	Set OneAgent resources for cloudNativeFullStack	`requests: {cpu: 100m, memory: 256Mi}`, `limits: {cpu: 300m, memory: 512Mi}`	Recommended	Sizing
61	Size ActiveGate by cluster node count	See ActiveGate Sizing Reference in Section 4	**Critical**	Sizing
62	Size OTel Collector by telemetry volume	Low: 200m/256Mi, Medium: 500m/512Mi, High: 1000m/1Gi, Very High: 2000m/2Gi	Recommended	Sizing
63	Size Log Monitoring by daily log volume	Low (<1GB): 200m/256Mi, Medium (1-10GB): 500m/512Mi, High (10-50GB): 1000m/1Gi	Recommended	Sizing
64	Monitor OneAgent memory by absolute usage (no limits by default)	OneAgent runs without resource limits; use `dt.kubernetes.container.memory_working_set` with `matchesValue(k8s.container.name, "dynatrace-oneagent")`	Recommended	Sizing

12. Security and Secrets

#	Best Practice	Recommended Setting/Value	Priority	Category
65	Never store API tokens in Git	Use External Secrets Operator, Sealed Secrets, or SOPS	**Critical**	Security
66	Use External Secrets Operator for token management	`ExternalSecret` CR referencing Vault, AWS Secrets Manager, or GCP Secret Manager	Recommended	Security
67	Apply NetworkPolicies allowing Dynatrace egress on port 443	`egress: [{to: [{ipBlock: {cidr: 0.0.0.0/0}}], ports: [{protocol: TCP, port: 443}]}]`	Recommended	Security
68	Configure proxy via DynaKube spec (not env vars)	`spec.proxy.value: https://proxy.example.com:8080` or `spec.proxy.valueFrom.secretKeyRef`	Recommended	Security
69	Apply RBAC per namespace aligned with Dynatrace boundaries	K8s `namespace-admin` = full namespace visibility in Dynatrace	Recommended	Security

13. GitOps and Lifecycle

#	Best Practice	Recommended Setting/Value	Priority	Category
70	Manage DynaKube via GitOps (ArgoCD or Flux)	Store DynaKube YAML in Git, apply via GitOps controller	Recommended	Lifecycle
71	Use Kustomize overlays for environment-specific config	`base/dynakube.yaml` + `overlays/{dev,staging,prod}/` patches	Recommended	Lifecycle
72	Pin operator Helm chart version in ArgoCD/Flux	`targetRevision: 1.9.0` (ArgoCD) or `version: "1.9.x"` (Flux)	**Critical**	Lifecycle
73	Set `selfHeal: true` in ArgoCD for drift correction	`syncPolicy.automated.selfHeal: true`	Recommended	Lifecycle
74	Set `prune: false` for DynaKube in ArgoCD	Prevent accidental DynaKube deletion on Git removal	**Critical**	Lifecycle

#	Best Practice	Recommended Setting/Value	Priority	Category
75	Use Flux `dependsOn` to order operator before DynaKube	`dependsOn: [{name: dynatrace-operator}]`	Recommended	Lifecycle
76	Use ArgoCD sync waves for deployment ordering	Wave 0: Namespace, Wave 1: Operator, Wave 2: DynaKube	Recommended	Lifecycle
77	Use `helm upgrade --reuse-values` for operator upgrades	Preserves custom values during upgrade	Recommended	Lifecycle
78	Deploy to dev -> staging -> prod-canary -> prod	Progressive rollout strategy	Recommended	Lifecycle

14. Cluster Health Alerting

#	Best Practice	Recommended Setting/Value	Priority	Category
79	Alert on Node NotReady	Node condition != Ready for >5 min = Critical	**Critical**	Alerting
80	Alert on high node CPU	CPU > 85% for 15 min = Warning	Recommended	Alerting
81	Alert on high node memory	Memory > 90% for 10 min = Critical	**Critical**	Alerting
82	Alert on disk pressure	Disk > 85% = Warning	Recommended	Alerting
83	Alert on OOMKilled events	Any OOMKilled event = Warning	**Critical**	Alerting
84	Alert on CrashLoopBackOff	Any CrashLoopBackOff = Warning	Recommended	Alerting
85	Alert on FailedScheduling	Events > 5 in 10 min = Warning	Recommended	Alerting
86	Alert on volume mount failures	Any FailedMount or FailedAttachVolume = Critical	**Critical**	Alerting
87	Monitor Dynatrace component health (OA, AG, CSI)	Track restarts, OOMKills, and data gaps for `dynatrace-oneagent` and `activegate` containers	**Critical**	Alerting
88	Target CPU utilization >40% avg for cost efficiency	Right-size if avg CPU usage <40%	Recommended	Cost
89	Target memory utilization >50% avg for cost efficiency	Right-size if avg memory usage <50%	Recommended	Cost

15. Workload and Application Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
90	Alert on container CPU usage > 85% sustained	Monitor `dt.kubernetes.container.cpu_usage` vs limits	Recommended	Workload
91	Alert on container memory usage > 90%	Monitor `dt.kubernetes.container.memory_working_set` vs limits	**Critical**	Workload
92	Monitor CPU throttling	`dt.containers.cpu.throttled_time > 0` indicates under-provisioned CPU limits	Recommended	Workload
93	Track service P99 latency and error rate	SLO: P99 < 500ms, error rate < 0.1%	Recommended	Workload
94	Filter on `isNotNull(field)` before aggregating optional fields	Prevent null group-by keys in DQL	Recommended	Workload
95	Use `arrayAvg()` before filtering/sorting timeseries results	`timeseries` returns arrays; convert to scalar first	**Critical**	DQL

16. Specialized Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
96	Instrument NGINX Ingress via ConfigMap `main-snippet`	`load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-x86-64/liboneagentnginx.so;`	Recommended	NGINX
97	NGINX instrumentation requires x86-64 and OneAgent 1.227+	No ARM support; pod name must contain `ingress-nginx-`	Recommended	NGINX
98	Enable Prometheus scraping for custom metrics	Annotate pods with `metrics.dynatrace.com/scrape: "true"`, `/port`, `/path`	Recommended	Prometheus
99	ActiveGate must be in-cluster for annotation-based Prometheus scraping	External ActiveGate cannot reach pod endpoints	**Critical**	Prometheus
100	Use Kpow + Prometheus scraping for Kafka consumer lag monitoring	`PROMETHEUS_EGRESS: "true"` in Kpow config; scrape annotations on Kpow pods	Optional	Kafka

17. Troubleshooting Practices

#	Best Practice	Recommended Setting/Value	Priority	Category
101	Check DynaKube status first	`kubectl -n dynatrace get dynakube` — expect `Running` phase	**Critical**	Troubleshooting
102	Verify all Dynatrace pods are running	`kubectl -n dynatrace get pods` — no CrashLoopBackOff or Error	**Critical**	Troubleshooting
103	Check OneAgent connectivity to tenant	`kubectl -n dynatrace exec <oneagent-pod> -c oneagent -- curl -v https://<tenant>.live.dynatrace.com/api/v1/time`	Recommended	Troubleshooting
104	Check namespace labels when injection fails	`kubectl get namespace <ns> -o jsonpath='{.metadata.labels}'` must match DynaKube `namespaceSelector`	Recommended	Troubleshooting
105	Verify init container present for injected pods	`kubectl get pod <pod> -o jsonpath='{.spec.initContainers[*].name}'`	Recommended	Troubleshooting
106	Check webhook registration	`kubectl get mutatingwebhookconfigurations` — Dynatrace webhook must exist	Recommended	Troubleshooting
107	Collect support bundle for escalation	DynaKube YAML, pod YAML, events, operator logs, OneAgent logs in a tar.gz	Optional	Troubleshooting
108	Use DQL to detect Dynatrace component failures across fleet	Query `events` for `dynatrace` + `Failed/OOMKilled/BackOff`	Recommended	Troubleshooting
109	Detect metric data gaps with timeseries count queries	`timeseries cpuPoints = count(dt.host.cpu.usage)` then `filter minPoints == 0`	Recommended	Troubleshooting

Summary

This notebook contains **109 actionable best practices** across 17 categories for Dynatrace Kubernetes monitoring:

Category	Count	Key Takeaway
Deployment Mode	5	Use `cloudNativeFullStack`; never `classicFullStack` for new deployments
Operator Installation	5	Helm OCI install with CSI driver enabled
DynaKube Core	7	v1beta5/v1beta6, tolerations, resource limits
ActiveGate	5	`kubernetes-monitoring` + `routing`, 2+ replicas in prod
Namespace/Injection	8	`namespaceSelector` for scope control, ResourceQuotas on every namespace
Feature Flags	7	Version detection, K8s app detection, injection failure policy
Metadata Enrichment	7	Settings-based only, never mix with pod annotations
Log Monitoring	6	Empty `logMonitoring: {}`, resources under `templates`
OTel/Telemetry Ingest	6	Pin version, StatsD via OTel Collector only
CSI Driver	3	Always set provisioner limits
Resource Sizing	5	Size by cluster/volume tier
Security	5	Never store tokens in Git, use ESO
GitOps/Lifecycle	9	Pin versions, progressive rollouts, prune: false for DynaKube
Cluster Alerting	11	Node health, OOMKill, CrashLoop, Dynatrace component health
Workload Monitoring	6	CPU throttling, P99 SLOs, `arrayAvg()` for timeseries
Specialized Monitoring	5	NGINX via ConfigMap, in-cluster AG for Prometheus
Troubleshooting	9	DynaKube status, connectivity, DQL-based diagnostics

---

References

- DynaKube parameters (DT docs)
- DynaKube feature flags (DT docs)
- Set up Dynatrace on Kubernetes (DT docs)
- How Dynatrace Kubernetes monitoring works (DT docs)
- Quickstart for K8s setup (DT docs)
- Full observability deployment (DT docs)
- Kubernetes log monitoring (DT docs)
- Metadata enrichment (DT docs)
- Kubernetes app (DT docs)
- Operator + DynaKube troubleshooting (DT docs)
- Dynatrace Operator Helm chart (Dynatrace GitHub)
- Dynatrace Operator releases (Dynatrace GitHub)
